GRADE 6 · CODING CLUB · LESSON 2

Pydantic

Data with rules: teaching Python to check information

Like a form that won't let you write "banana" in the age box.

TODAY

1. Why check data?
2. Models: a form with rules
3. Types
4. Friendly error messages
5. Extra rules with Field
6. Build: Monster Cards
7. Quiz

Why check data?

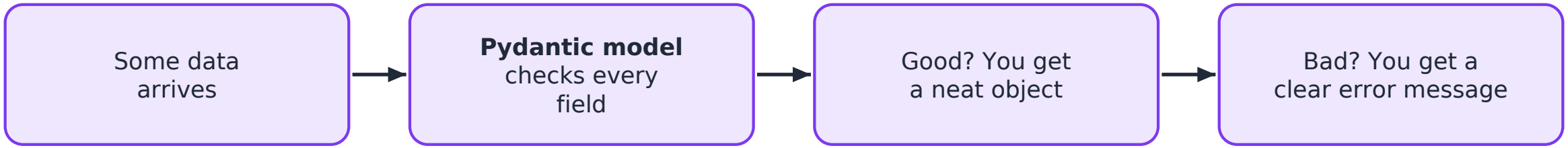
- Imagine a sign-up form where someone types their age as **"banana"**
- Or a game where a player's score is **-1000000**
- Or a pet that is **500 years old**
- Bad data causes **bugs**, **crashes** and sometimes **security problems**
- So good programs **check data before using it** — this is called **validation**
- **Pydantic** is a free Python library that does the checking for us

TALKING POINTS

Remember the Checker from the Human API game? Pydantic is the Checker. Ask kids for funny examples of bad data they could type into a form. Then ask: what could go wrong if a program believed it?

A model is a form with rules

We write a **model** once — it describes what **good** data looks like



Think of a school trip form: **Name** must be filled in · **Age** must be a number · **Permission** must be yes or no. The office checks the form. Pydantic is the office!

TALKING POINTS

The word 'model' here means a description or template, like a model of a house. Hold up a real paper form to make it concrete.

My first Pydantic model

`str` = text · `int` = whole number · `float` = decimal · `bool` = True/False

```
from pydantic import BaseModel

class Pet(BaseModel):
    name: str            # text
    age: int             # whole number
    is_friendly: bool = True    # yes/no (default True)

biscuit = Pet(name="Biscuit", age=3)
print(biscuit)
print(biscuit.name, "is", biscuit.age)
```

Output:

```
name='Biscuit' age=3
is_friendly=True

Biscuit is 3
```

is_friendly is **True** because that's the **default**

TALKING POINTS

Type it together line by line. The colon-type part (name: str) is called a type hint — it tells Pydantic what kind of data to expect. Kids can run this in Thonny's shell.

Pydantic is helpful… and strict!

If it can safely fix the data, it does. If it can't, it tells you **exactly** what's wrong.

```
Pet(name="Biscuit", age="3")
# OK!  name='Biscuit' age=3 is_friendly=True
#      Pydantic turned the text "3" into the number 3

Pet(name="Biscuit", age="three")
# ValidationError: 1 validation error for Pet
# age
#   Input should be a valid integer, unable to parse
#   string as an integer [input_value='three']
```


TALKING POINTS

These outputs are real — run them live. Read the error out loud: it says WHICH field (age), WHAT was wrong, and WHAT you typed. Ask: is this more helpful than a program just crashing? That's why people love Pydantic.

Extra rules with Field

`ge` = greater than or equal · `le` = less than or equal · `gt`, `lt` = greater / less than

```
from pydantic import BaseModel, Field

class Student(BaseModel):
    name: str = Field(min_length=1, max_length=20)   # not empty, not too long
    age: int = Field(ge=5, le=18)                    # 5 to 18
    favourite_colour: str | None = None              # optional

Student(name="", age=40)
# 2 validation errors for Student
# name:  String should have at least 1 character
# age:   Input should be less than or equal to 18
```


TALKING POINTS

Pydantic finds ALL the mistakes at once, not just the first. 'str | None = None' means the field is optional — you can leave it out. Challenge: what rule would stop a negative score in a game? (score: int = Field(ge=0)).

Turning a model back into data

Remember JSON from Lesson 1? Pydantic can turn models into JSON — exactly what an API sends back

```
rex = Pet(name="Rex", age=2)

rex.model_dump()
# {'name': 'Rex', 'age': 2, 'is_friendly': True}      <- a Python dictionary

rex.model_dump_json()
# '{"name":"Rex","age":2,"is_friendly":true}'          <- JSON text!
```


TALKING POINTS

This links the two lessons: APIs send JSON, and Pydantic makes JSON from models. Next lesson, FastAPI will do this for us automatically.

Build: Monster Cards (25 minutes)

- Invent your own monster card game! Make a `Monster` model with:
  - **name**: text, 1 to 15 letters
  - **power**: whole number from 1 to 100
  - **element**: text, e.g. "fire", "water"
  - **can_fly**: True or False, default False
- Make **3 good monsters** and print them
- Then try to **break** your model: power 500? an empty name? power "lots"?
- Read each error message and **fix** the data
- Stretch: add **moves** (a list of text) — try `moves: list[str] = []`

TALKING POINTS

Walk around and ask kids to read their error messages aloud and explain them in their own words. Stretch idea: use Literal["fire", "water", "earth", "air"] for element (from typing import Literal) so only those four are allowed.

Quiz time!

- 1. What does Pydantic do?
- 2. What is a **model**?
- 3. What happens with `Pet(name="Max", age="7")`? And `age="seven"`?
- 4. Write a rule so a game score can never be negative.
- **Next lesson:** we make our own **web API** with **FastAPI**!

TALKING POINTS

Answers: 1) Checks (validates) data using rules we write. 2) A description of what good data looks like — a form with rules. 3) "7" becomes the number 7; "seven" gives a validation error. 4) score: int = Field(ge=0).